

Mycelium 4.0

Rapport de pré-étude et spécifications

Arno Lécivain - Mohamed Lahmamsi - Mohamed Allay - Elise Bottois - Thibault
Dufourcq

Encadrants : PARLAVANTZAS Nikos - PAROL-GUARINO Volodia

Avec la participation de : MOUREAU Julien

2024/2025



Contents

1	Introduction	3
1.1	Contexte	3
1.2	Historique des versions	3
1.3	Objectifs de Mycélium 4.0	4
2	Pré-étude	4
2.1	Technologies en place	4
2.2	Pré-étude des versions précédentes	5
3	Architecture logicielle et matérielle existante	7
3.1	Architecture matérielle	7
3.2	Architecture logicielle	9
3.2.1	Structure et orchestration du cluster de Raspberry	10
3.2.2	Traitement des données, stockage et visualisation	11
3.3	Exemple de fonctionnement sur un scénario du Mycelium 3.0	12
4	Spécifications générales et fonctionnelles	13
4.1	Intégration d'une étude précédente Mycelium (Inondation)	13
4.1.1	Adaptation à l'environnement de travail (VM)	13
4.1.2	Adaptation pour le cluster de Raspberry Pi	13
4.1.3	Fonctionnement avec les datasets locaux	14
4.1.4	Fonctionnement avec des données en temps réel	14
4.2	Scénario de déconnexion du cluster de Raspberry Pi	14
4.2.1	Implémentation d'un proxy pour la détection de perte de connexion	14
4.2.2	Implémentation d'une base de données légère sur le cluster avec 2 buckets InfluxDB	14
4.2.3	Compression des données du bucket d'erreur	15
4.2.4	Détection du retour de connexion et envoi des données	15
4.2.5	Mode de Fonctionnement Allégé des Fonctions VPS sur le Cluster	16
4.3	Gestion dynamique de la fréquence des capteurs	17
4.3.1	Gestion dynamique des capteurs en cas d'événements	17
4.3.2	Appel à des fonctions prédictives via un proxy	17
4.3.3	Exécution des fonctions de prévision allégées	18
4.3.4	Schéma du processus	18
4.4	Mise en place d'un environnement virtuel simple	18
4.5	Le proxy dans Mycelium 4.0	20
5	Conclusion	22

1 Introduction

1.1 Contexte

Le projet Mycélium 4.0 s'inscrit dans le projet national de recherche TERRA FORMA qui vise une meilleure compréhension des environnements et de leur évolution en particulier au regard de l'activité humaine. Ce projet, coordonné par Laurent Longuevergne, prend la forme de déploiement de capteurs environnementaux intelligents dans des environnements pertinents à suivre. Des sites ainsi équipés existent déjà : les 3 pilotes se trouvent respectivement dans les Alpes, le Gers et le Morbihan, et 12 autres sont en cours d'équipement, dont la zone sur laquelle se penche notre projet.

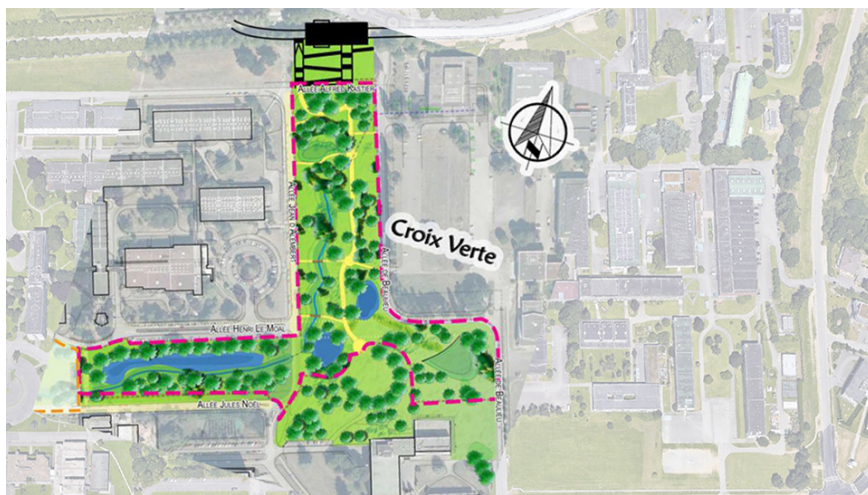


Figure 1: Représentation cartographique de la zone de la Croix Verte sur le campus de Beaulieu

C'est en parallèle de la construction de la ligne du métro B que Mycélium voit le jour en 2021, afin d'assurer le suivi de la renaturation de la zone de la Croix Verte à Beaulieu (ci-dessus). En effet, compte tenu des conséquences sur l'environnement du chantier de construction de la ligne B, à savoir la perturbation de la biodiversité locale avec le défrichement d'espaces verts et l'abattage de nombreux arbres, une contrepartie a été mise en place. Mycélium fait donc partie intégrante de cette dernière, qui consiste en la renaturation d'espaces (création de mares, plantation d'arbustes...) et la gestion et le suivi de ces espaces.

L'équipement du site permet la collecte de données de suivi cruciales à propos de l'évolution de l'écosystème et donc également des ressources naturelles face aux changements environnementaux. C'est également un outil de test pour des déploiements à de plus grandes échelles dans d'autres projets.

1.2 Historique des versions

En 2021, le lancement du projet dans sa première version **Mycélium 1.0**, vise à déployer un réseau de capteurs intelligents à basse consommation sur le site de la Croix Verte. Un seul nœud de capteurs est d'abord déployé, et cette première version pose les bases de l'architecture globale du projet qui est encore d'actualité malgré certaines évolutions, à savoir l'architecture Fog Computing notamment et l'utilisation du protocole de communication LoRaWAN, technologies qui seront définies par la suite dans la section [2.1 Technologies en place](#).

Mycélium 2.0 prend la suite de la première version du système avec l'objectif de l'améliorer. Dans ce cadre, en plus de l'ajout de nouvelles fonctionnalités et de nouveaux scénarios; un focus a été fait sur l'amélioration de la gestion de données à grande échelle qui a permis de booster des performances.

Mycélium 3.0, refonte globale du projet, a œuvré sur différents axes pour optimiser et pérenniser sa manipulation. En premier lieu en implémentant des moyens de prise en main du projet tels que la réalisation d'un guide et la mise en place d'une machine virtuelle, et également en simplifiant l'architecture du système global par des substitutions de technologies et l'ajout de scénarios faisant appel à des données externes, provenant d'OSUR ou encore de Météo France.

1.3 Objectifs de Mycélium 4.0

Ces versions successives mènent à Mycélium 4.0 qui va focaliser les efforts sur des scénarios résiliants aux déconnexions avec les serveurs centraux (coupure internet, crash...) et de nouveaux scénarios incorporant un retour d'information pour contrôler les capteurs automatiquement. Cela va passer par la variabilité de la fréquence d'échantillonnage des capteurs ou encore par l'implémentation d'un proxy pour détecter une éventuelle perte de connexion, solutions que nous allons exposer plus en détail dans la partie 4. Nous allons également favoriser une virtualisation (VM) sur Nix, qui est un gestionnaire de paquets pour système Linux, rendant notamment la gestion de l'environnement de développement plus fiable et reproductible.

La pré-étude du projet, au regard du détail des technologies et également des versions précédentes du projet Mycélium, sera détaillée dans la section 2. Dans un second temps, l'architecture générale du projet dans son état actuel sera présentée en section 3, pour finir avec la section 4 qui introduira les spécifications générales et fonctionnelles de Mycélium 4.0.

2 Pré-étude

2.1 Technologies en place

L'architecture du projet repose sur la combinaison de différentes technologies assurant le déploiement d'un système de contrôle du réseau de capteurs. Ces technologies sont articulées comme suit : les mesures sont d'abord récoltées par les capteurs basse consommation sur le site de la Croix Verte puis agrégées pour être transférées à l'aide du protocole **LoRaWAN**. Ce protocole de communication radio envoie les données sur une fréquence de 868 MHz, offrant une portée de plusieurs kilomètres et garantissant une consommation énergétique très réduite tout en assurant une transmission fiable d'un faible volume de données, le rendant idéal pour la solution.

Une fois transmises à une gateway LoRaWAN, les données sont alors dirigées vers un nœud de calcul proche du site, selon l'infrastructure Fog Computing. Le **Fog Computing** est une variante de l'infrastructure de cloud computing traditionnel qui permet une meilleure gestion des calculs à l'aide d'une couche supplémentaire entre les objets connectés et le cloud. Cette couche de nœuds plus proches du terrain effectue certaines tâches de pré-traitement : cela permet d'y optimiser les analyses plus urgentes, et de réserver les plus complexes et approfondies au cloud.

Dans la solution, les nœuds de calcul prennent la forme d'un cluster de **Raspberry Pi**. Ce dernier est un micro-ordinateur compact, économique et doté de ports divers et multiples lui permettant de se connecter à des périphériques et capteurs variés, idéal pour des applications de domotique ou de suivi environnemental. Cela fait de lui une solution populaire pour les projets de calculs distribués et d'IoT, où il fonctionne comme un nœud intelligent idéal en Fog Computing comme dans le cas présent.

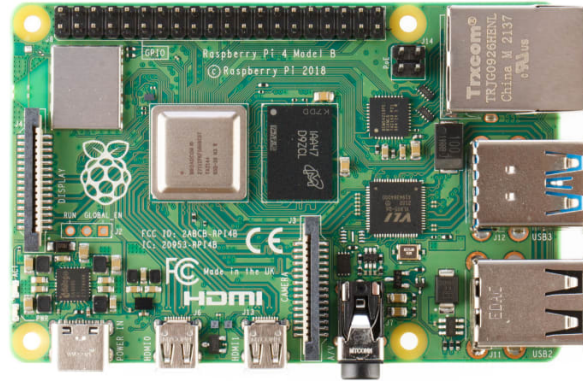


Figure 2: Photographie d'une carte Raspberry Pi

Ces noeuds permettent un prétraitement local et indépendant de la connexion vers un serveur central, en utilisant le type de service **Function as a Service (FaaS)** qui repose sur le modèle Serverless et décharge les développeurs de la gestion de l'infrastructure sous-jacente. Dans notre architecture, les données sont traitées par des fonctions OpenFaaS, organisées selon le modèle FaaS dans lequel chaque fonction exécute une tâche spécifique. Ces fonctions assurent divers traitements, tels que le filtrage des données, le déclenchement de notifications basées sur des critères prédéfinis, ou encore des analyses en temps réel. Cette approche modulaire offre une grande flexibilité et simplifie la maintenance de l'architecture, chaque fonction pouvant être gérée indépendamment. FaaS permet donc de supporter le prétraitement de calculs en fonction de leur urgence ou complexité, ce qui optimise le traitement et la transmission dans l'architecture du projet.

Les données sont stockées à l'aide de la base de données orientée série temporelle **InfluxDB** qui est incluse dans le cluster. Écrite en Go pour un stockage performant, elle est en particulier efficace pour les opérations de surveillance et d'analyse en temps réel notamment en capteurs IoT. C'est donc parfaitement approprié pour notre projet : en effet, tout en stockant les valeurs enregistrées, elle permet également de conserver les métriques de performance du cluster (CPU, mémoire...).

Kubernetes, système open source conçu pour automatiser la gestion et le déploiement des applications conteneurisées sur des clusters, intervient également dans la solution, afin de faciliter l'utilisation de nos clusters de Raspberry Pi.

Enfin, c'est le protocole de communication **MQTT** qui assure la transmission entre les noeuds et les serveurs centraux. Ce dernier est un protocole de messagerie basé sur le principe publication/abonnement à des topics qui est beaucoup utilisé pour la communication de machine à machine (capteurs intelligents, IoT...) et supporte la messagerie entre appareil et cloud. Ces caractéristiques en font un protocole idéal pour le système Mycélium, d'autant plus que MQTT est léger et efficace, ce qui permet l'optimisation de la bande passante.

Afin de comprendre davantage l'articulation de ces technologies entre elles au sein du projet, il est pertinent de se référer à la section 3, qui détaille davantage l'architecture du projet avec des schémas clairs.

2.2 Pré-étude des versions précédentes

Mycélium 1.0

La première version du projet était axée sur la création de scénarios, dont voilà les événements observés qui ont été implémentés :

- **Scénario “Luminosité anormale : Intempéries”** : vérifie que la luminosité diminue lorsqu’il pleut.
- **Scénario “Luminosité anormale selon l’heure”** : vérifie que la luminosité est cohérente avec l’heure.
- **Scénario “Neige”** : regarde si le pluviomètre se remplit après une baisse de la température en dessous de 0°C.
- **Scénario “Températures extrêmes”** : qui notifie lorsque les températures sont extrêmement basses ou extrêmement élevées.
- **Scénario “Températures anormales selon l’historique de températures”** : vérifie que les températures n’augmentent pas ou ne diminuent pas trop vite.
- **Scénario “Vol et perte de matériel”** : notifie s’il y a un mouvement rapide du capteur, qui pourrait signifier un vol.
- **Scénario “Capteurs endommagés”** : notifie si les données mesurées sont aberrantes.

Trois nouvelles fonctions pour ces scénarios ont ensuite été développées et le cluster des Raspberry Pi a été déployé pour traiter les données envoyées par le nœud Solo via le protocole LoRaWAN. De plus, les membres du premier groupe ont permis le traitement puis la visualisation de ces données en créant des fonctions OpenFaaS.

Mycélium 2.0

Le second groupe, après s’être approprié le projet, l’a amélioré d’abord à l’aide de l’ajout d’un lien dans le cloud. Cela a été entrepris afin de déléguer certains traitements et éviter une surcharge, étant donné les ressources limitées du cluster qui pouvaient rapidement être saturées. Trois nouveaux scénarios ont également été ajoutés à la 1ère version :

- **Scénario “Normales saisonnières”** : notifie lorsque les températures détectées sortent des normales saisonnières.
- **Scénario “Orage”** : notifie la présence d’épisodes orageux.
- **Scénario “FoxyFind”** : notifie la présence d’animaux devant une caméra.

Mycélium 3.0

Le troisième groupe a opéré quelques modifications, telles que le changement de certaines fonctions Python en Go pour gagner en espace, ou encore la réception des données qui a été enrichie avec le broker de l’OSUR en plus du nôtre. Cette version a aussi permis la mise en place d’une documentation exhaustive qui a déjà prouvé son utilité, ainsi que l’introduction d’une machine virtuelle qui émule le comportement d’un cluster et permet d’appeler les fonctions OpenFaaS.

Application Mycélium

En parallèle du déploiement de la solution Mycélium, un groupe d’étude pratique de 3ème année a travaillé l’année dernière sur le développement d’une application de suivi environnemental étroitement liée à notre projet. L’objectif de l’application est de formuler des prédictions basées sur l’analyse des données remontées par les capteurs implantés sur le site. La dernière version de ce projet a abouti au développement du back-end de l’application qui est à même de diffuser des alertes en cas d’inondation.

3 Architecture logicielle et matérielle existante

L'architecture de Mycelium 4.0 repose sur les fondations redéfinies par la version précédente, Mycelium 3.0, qui a marqué un tournant majeur dans la conception de ce projet. La version 3.0 a permis de remodeler l'architecture du projet, visant à simplifier sa structure, à améliorer les performances et à réduire la consommation énergétique. Cette réorganisation a également rendu l'ensemble du système plus simple et plus accessible.

Pour Mycelium 4.0, il est essentiel de bien maîtriser cette architecture afin d'avancer efficacement et de bâtir sur ses bases solides. Nous commencerons donc par présenter l'architecture globale du projet de l'année précédente, afin de poser les repères essentiels pour le développement de cette nouvelle version.

3.1 Architecture matérielle

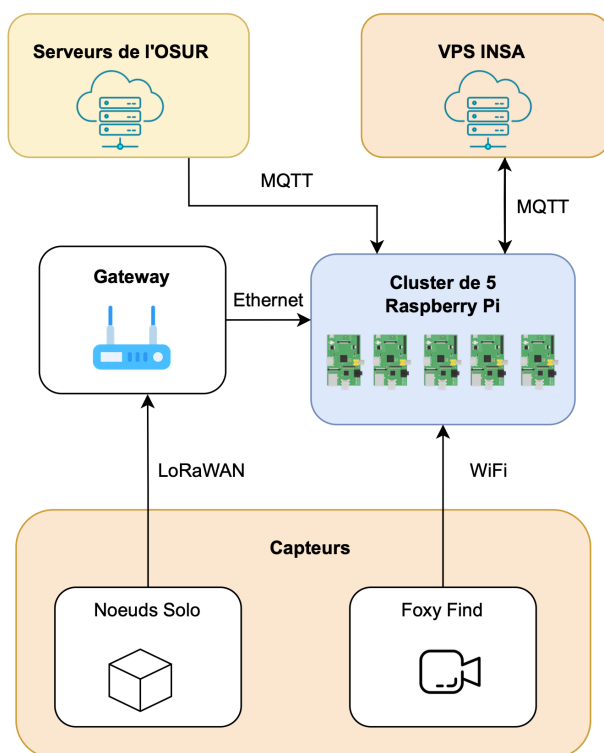


Figure 3: Schéma de l'architecture matérielle de Mycelium 3.0

L'architecture matérielle de Mycelium, illustrée en figure 1, repose sur un réseau de capteurs déployés dans la zone de la Croix-Verte, conçu pour collecter des données environnementales variées. Les capteurs, appelés nœuds SoLo, sont des dispositifs autonomes utilisant la technologie LoRaWAN pour transmettre leurs informations. Ce protocole de communication radio envoie les données sur une fréquence de 868 MHz, offrant une portée de plusieurs kilomètres et garantissant une consommation énergétique très réduite tout en assurant une transmission fiable d'un faible volume de données.

Chaque nœud SoLo est protégé par un boîtier étanche contenant plusieurs capteurs, tels que des capteurs de température, d'humidité, de luminosité, d'accélération, de pression et de pluviométrie. La configuration de ces capteurs est facilement ajustable grâce à des fichiers JSON, permettant de personnaliser la fréquence d'envoi et de mesure ainsi que les paramètres réseau. En complément de ces nœuds SoLo, l'OSUR fournit

un accès à ses capteurs répartis sur le campus. Un port sécurisé a été ouvert pour permettre aux adresses IP de l'INSA de recevoir les nouvelles données de ces capteurs, établissant ainsi une connexion directe entre le cluster de Mycélium et les serveurs de l'OSUR. Mycélium intègre également un module de détection de faune, nommé Foxy Find, qui, équipé d'une caméra, repère le passage des renards et transmet ces informations via WiFi.

Pour le projet, ces capteurs transmettent les données collectées vers une gateway installée dans les locaux de l'INSA. La gateway LoRaWAN agit comme un pont entre les nœuds SoLo et le réseau IP, rassemblant les informations envoyées par les capteurs et les transférant via une connexion Ethernet au cluster de Raspberry Pi. Ce cluster, situé dans le bâtiment INFO, se compose de cinq Raspberry Pi dotés de 2 Go de RAM chacune. L'une des Raspberry joue le rôle de superviseur, tandis que les quatre autres sont des nœuds contrôleurs qui reçoivent les directives de ce superviseur. L'objectif de ce cluster est de traiter et analyser les données issues des capteurs pour en extraire des informations utiles.

Cependant, le cluster est limité par une capacité de mémoire vive restreinte, ce qui peut constituer un frein pour certains traitements intensifs. Pour pallier cette limitation, un VPS hébergé par l'INSA a été ajouté au système. Ce serveur virtuel peut prendre en charge une partie des calculs lorsque les ressources du cluster sont insuffisantes. Ainsi, lorsque la charge du cluster dépasse un certain seuil, les opérations de traitement sont transférées sur le serveur de l'INSA, garantissant une efficacité et une réactivité accrues.

3.2 Architecture logicielle

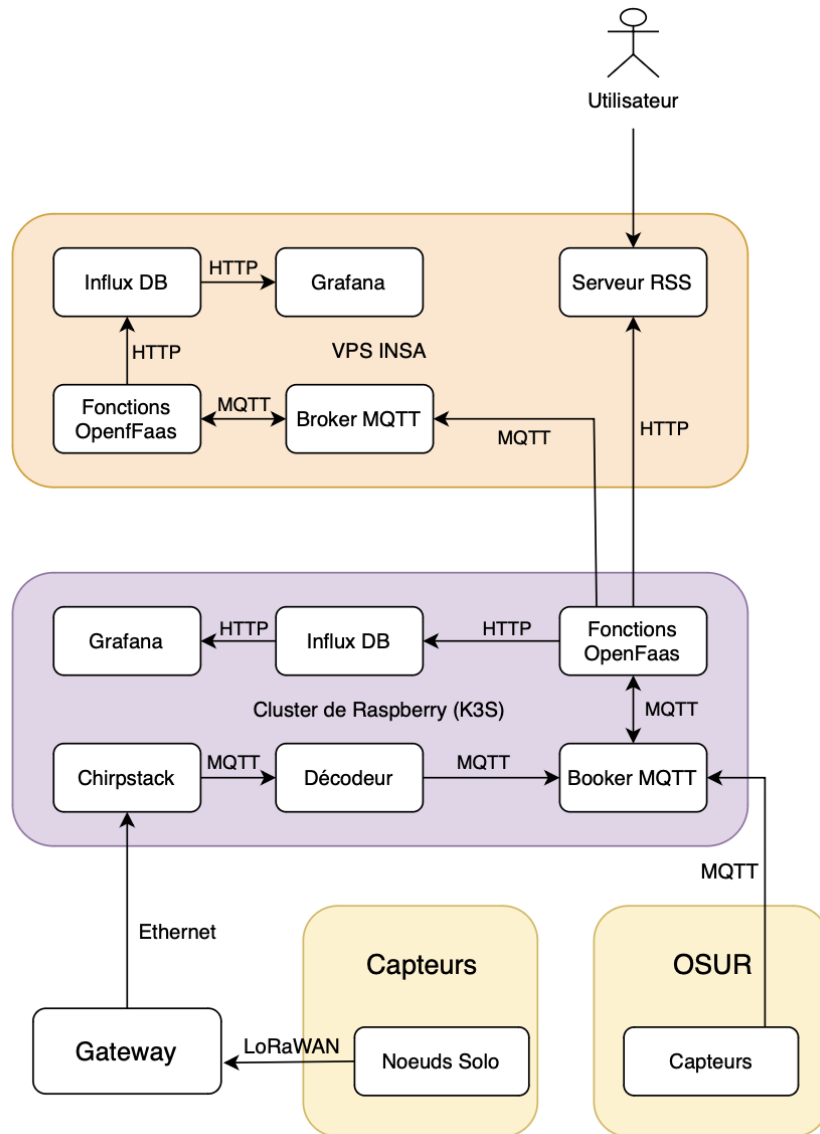


Figure 4: Schéma de l'architecture logicielle de Mycelium 3.0

3.2.1 Structure et orchestration du cluster de Raspberry



Figure 5: Cluster de Raspberry

L'architecture logicielle de Mycelium 3.0 repose sur un cluster de cinq Raspberry, comme on peut le voir sur la figure 5, dont un est désigné comme le nœud Contrôleur et les quatre autres comme des nœuds travailleurs, assurant une séparation du plan de contrôle et de données. Ce cluster, couplé à un serveur VPS situé à l'INSA, est conçu pour traiter des données en provenance de capteurs et répondre aux besoins de temps réel tout en offrant une résilience face aux pics de charge et une consommation minimale en l'absence de traitement nécessaires. L'objectif est de faciliter l'utilisation de ce cluster comme s'il s'agissait d'une seule machine, grâce au système Kubernetes qui assure la gestion des conteneurs. Ces conteneurs encapsulent les applications avec leur code, leurs paramètres, bibliothèques, et dépendances, permettant ainsi une exécution homogène et compatible dans différents environnements.

Kubernetes, en plus de gérer l'installation et la mise à jour des conteneurs, en assure l'échelle et la résilience en répartissant automatiquement les charges de travail sur les différents nœuds enfants et en redéployant les conteneurs en cas de panne d'un nœud, garantissant ainsi la continuité du service. Le système est optimisé pour traiter des données collectées sur le terrain par des capteurs SoLo, qui utilisent le protocole LoRaWAN, évoqué précédemment. Ces capteurs communiquent avec une gateway LoRaWAN située à l'INSA, qui reçoit les données et les transmet ensuite au cluster de Raspberry Pi pour un traitement initial.

3.2.2 Traitement des données, stockage et visualisation

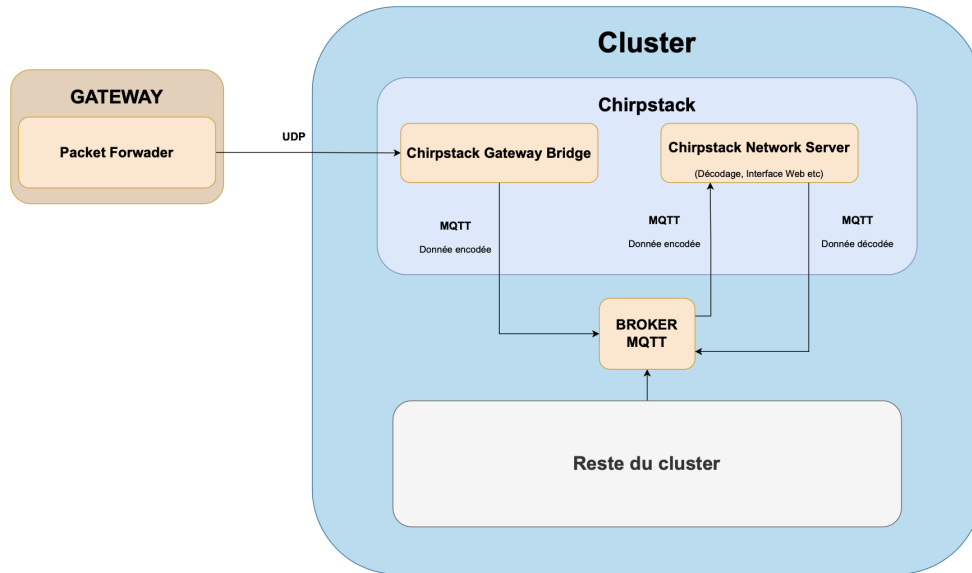


Figure 6: Schéma de l'architecture matérielle de Mycelium 3.0

Au cœur de l'architecture, le cluster de Raspberry Pi, basé sur Kubernetes (K3S), joue un rôle central dans la gestion et le traitement des données en amont. Ce cluster héberge plusieurs services pour assurer le bon fonctionnement de l'ensemble du système. Le composant principal de gestion des données LoRaWAN est Chirpstack, un réseau de serveurs open-source conçu pour faciliter l'intégration de capteurs utilisant LoRaWAN. Chirpstack est responsable de la réception et du décodage des données brutes transmises par les capteurs. Il est structuré autour de plusieurs sous-composants, notamment le Chirpstack Gateway Bridge qui reçoit les données de la gateway LoRaWAN, les transforme en un format exploitable, puis les publie sur un topic MQTT via le protocole MQTT. Le Chirpstack Network Server assure ensuite le suivi des capteurs et des passerelles, en publiant les données décodées sur un autre topic MQTT, rendant ces données immédiatement accessibles aux autres services du cluster via le broker MQTT.

Le broker MQTT centralise la distribution des données vers différents modules, permettant ainsi une communication fluide entre les composants de l'architecture. Les données brutes ou partiellement traitées sont transmises à une fonction de décodage dédiée, qui transforme les messages en informations lisibles. Une fois décodées, les données sont traitées par des fonctions OpenFaaS. Ces fonctions réalisent divers traitements : filtrage de données, déclenchement de notifications selon des critères prédéfinis, et analyses en temps réel. Cette organisation modulaire assure une grande flexibilité et facilite la scalabilité de l'architecture, car chaque fonction peut être gérée indépendamment.

Pour le stockage et le monitoring, le cluster inclut également InfluxDB, qui permet de stocker les valeurs enregistrées par les capteurs et qui conserve des métriques de performance du cluster (CPU, mémoire, etc.). Ces données sont visualisées dans Grafana, un tableau de bord interactif permettant aux opérateurs de surveiller l'état du cluster en temps réel et de détecter les anomalies.

En cas de surcharge du cluster ou si des traitements plus lourds sont nécessaires, une communication est établie entre le cluster de Raspberry Pi et un VPS hébergé à l'INSA. Cette communication repose également sur MQTT pour permettre un échange rapide et fiable des données. Le VPS reprend des responsabilités

similaires au cluster, mais joue également un rôle de soutien pour délester le cluster des tâches intensives. Lorsqu'il reçoit les données, le VPS les stocke de manière centralisée et durable dans une autre instance d'InfluxDB, permettant ainsi de conserver un historique des données de mesure pour des analyses ultérieures. Le VPS héberge également une instance d'OpenFaaS pour exécuter des traitements supplémentaires sur les données, notamment des calculs intensifs ou des agrégations plus complexes qui seraient trop coûteuses à exécuter directement sur le cluster de Raspberry Pi. Ce modèle de calcul déporté permet d'avoir une garantie que les données restent accessibles même en cas de défaillance partielle du cluster ou en cas de déconnexion. En outre, le VPS intègre un serveur RSS, qui permet de distribuer des notifications aux utilisateurs lorsque des scénarios spécifiques se déclenchent, par exemple en cas de dépassement d'un seuil critique ou de détection d'un événement inhabituel dans les données. Ce serveur RSS facilite l'accès des informations pertinentes aux utilisateurs, leur offrant ainsi une manière pratique de rester informés des événements importants.

Enfin, pour compléter cette architecture et offrir une vision intégrée de l'ensemble des données, des données supplémentaires peuvent être récupérées auprès des serveurs de l'OSUR via MQTT. Cela permet d'obtenir des informations environnementales contextuelles en complément des données des capteurs Mycelium3.0, enrichissant ainsi l'analyse et la compréhension des phénomènes observés.

3.3 Exemple de fonctionnement sur un scénario du Mycelium 3.0

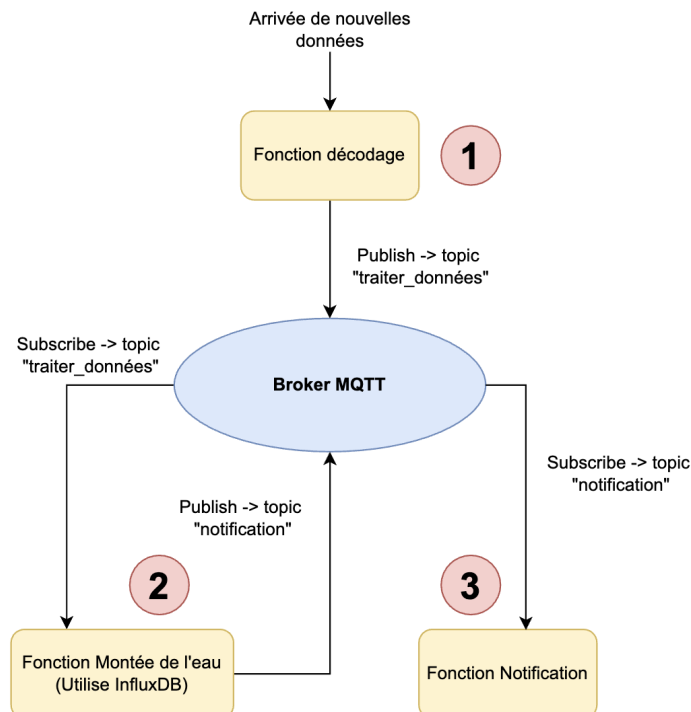


Figure 7: Schéma du fonctionnement du scénario "Montée des eaux"

Dans cette section, nous allons expliquer le fonctionnement de l'architecture sur le scénario "Montée des eaux", développé par le groupe du projet Mycelium 3.0 l'année dernière.

Les données collectées par le capteur sont d'abord traitées par une fonction de décodage, qui interprète les informations brutes reçues et les convertit dans un format exploitable. Une fois décodées, ces données

sont publiées sur un topic dédié au sein d'un broker MQTT, nommé "traiterDonnées". Ce topic permet de centraliser les données prêtes à être traitées et rend possible la communication en temps réel avec les autres composants du système.

Des fonctions spécifiques sont alors abonnées à ce broker MQTT et surveillent le topic "traiterDonnées" en attente de nouvelles publications. Parmi elles, la fonction "Montée de l'eau" qui a pour mission de surveiller le niveau d'eau. Dès qu'une nouvelle donnée est disponible sur ce topic, elle est récupérée par cette fonction, qui effectue une comparaison avec la mesure précédente pour détecter toute augmentation significative. Si le niveau de l'eau a monté de plus de 5 cm depuis la dernière mesure, la fonction déclenche une alerte. Pour cela, elle consulte la base de données InfluxDB pour obtenir la dernière mesure de référence, ce qui garantit une comparaison précise avec la nouvelle donnée.

En cas d'alerte, la fonction "Montée de l'eau" publie un message sur un autre topic, nommé "notification", pour signaler une anomalie. Ce message de notification est alors pris en charge par la fonction "Notification", qui diffuse l'alerte au public via un flux RSS. Pour cela, la fonction envoie une requête POST vers le serveur RSS hébergé sur un VPS. Ce système permet ainsi un suivi automatisé et en temps réel des variations du niveau de l'eau, et cela notamment grâce aux fonctions OpenFaas.

4 Spécifications générales et fonctionnelles

Ce chapitre décrit les spécifications prévues pour le projet Mycelium dans le cadre de son évolution, avec une attention particulière sur les aspects fonctionnels et techniques nécessaires pour son adaptation et son déploiement. Les fonctionnalités proposées s'appuient sur une intégration et une amélioration des travaux précédents, tout en visant des solutions innovantes adaptées aux besoins actuels.

4.1 Intégration d'une étude précédente Mycelium (Inondation)

Dans le cadre de l'évolution de la solution Mycelium, il a été jugé pertinent d'exploiter les travaux réalisés par le groupe d'étude pratique de 3^e année sur une application de suivi environnemental. Comme mentionné dans la section 2.2, ce projet précédent visait à développer des outils permettant de formuler des prédictions et d'émettre des alertes en cas d'inondation.

4.1.1 Adaptation à l'environnement de travail (VM)

Dans l'environnement de test basé sur des machines virtuelles (VM), la communication initialement gérée via NATS (un système de messagerie distribué utilisé pour les communications asynchrones) sera remplacée par MQTT, un protocole léger adapté aux systèmes IoT. Parallèlement, les notifications actuelles via Discord seront intégrées dans le flux RSS existant, garantissant une continuité et une centralisation des notifications.

Enfin, des topics spécifiques seront configurés dans l'environnement VM pour simuler le transfert et la gestion des données, en prévision de leur déploiement sur l'infrastructure finale. L'utilisation des VM permettra ainsi de valider l'architecture modifiée et de garantir la robustesse des solutions avant leur mise en œuvre sur le cluster.

4.1.2 Adaptation pour le cluster de Raspberry Pi

Étant donné les différences entre l'environnement de travail virtuel et le cluster de Raspberry Pi, le projet sera ensuite configuré pour fonctionner sur ce dernier. La gestion réseau et les accès aux nœuds du cluster Raspberry Pi nécessiteront des ajustements pour garantir la compatibilité. Une architecture légère, basée sur K3S au lieu de Kubernetes, sera privilégiée pour optimiser l'utilisation des ressources limitées des Raspberry Pi tout en assurant une communication fluide entre les nœuds. Ces adaptations permettront d'exploiter le cluster pour le traitement des données en temps réel.

4.1.3 Fonctionnement avec les datasets locaux

Le projet utilise un modèle prédictif basé sur des réseaux de neurones récurrents de type LSTM (Long Short-Term Memory), adapté à l'analyse de séries temporelles complexes telles que les données hydrologiques et météorologiques. Initialement, ce modèle a été entraîné avec des données météorologiques australiennes et testé sur des données de Météo France. Il a pour objectif de prévoir des événements climatiques critiques, comme des inondations, en analysant des données collectées par des capteurs.

Cependant, pour garantir la pertinence des prédictions dans un contexte local, il sera nécessaire de réentraîner le modèle avec des données collectées spécifiquement à partir des capteurs de l'OSUR, si elles sont disponibles. Cette étape est essentielle, car les caractéristiques hydrologiques et climatiques locales, telles que celles du canal régional, diffèrent significativement des conditions observées en Australie.

Ce réentraînement permettra d'adapter le modèle aux spécificités climatiques locales et d'assurer une meilleure précision dans les prévisions.

4.1.4 Fonctionnement avec des données en temps réel

Enfin, des prédictions en temps réel seront mises en place à partir des données reçues directement des capteurs qui seront installés par Julien Moureau de l'OSUR. Le projet intégrera un flux continu de données provenant des capteurs connectés au réseau LoRaWAN, avec des prédictions de pluie réalisées périodiquement, par exemple, toutes les dix données reçues. Les nouvelles données seront également ajoutées au jeu d'entraînement du classifieur pour améliorer la précision des prévisions futures.

4.2 Scénario de déconnexion du cluster de Raspberry Pi

Ce scénario vise à garantir la continuité de la collecte et de la sauvegarde des données même en cas de perte de connexion entre le cluster de Raspberry Pi et le VPS, sans perte de données locales. En cas d'interruption de connexion, le système doit être capable de conserver temporairement les données dans une base de données locale, pour un transfert ultérieur vers le VPS dès que la connexion est rétablie.

4.2.1 Implémentation d'un proxy pour la détection de perte de connexion

Le proxy servira d'intermédiaire entre le VPS et le cluster, en centralisant tous les échanges. Son rôle principal est de détecter les échecs de communication et de gérer les tentatives de relance des transferts ultérieurement. Voici les fonctionnalités que le proxy aura :

- **Détection de perte de connexion** : Le proxy effectue des pings réguliers entre le cluster et le VPS. En cas d'échec, il signale une interruption de connexion.
- **Gestion des alertes** : Si la perte de connexion est confirmée, le proxy enverra une alerte, activant un mode de sauvegarde sur le cluster. Ce mode suspend les envois de données et lance les programmes de sauvegarde des données localement.

4.2.2 Implémentation d'une base de données légère sur le cluster avec 2 buckets InfluxDB

Référence sur InfluxDB : [2.1](#)

Une base de données légère sera mise en place sur le cluster pour stocker les données temporairement, avec deux buckets :

- **Bucket de sauvegarde** (dernières 24h) : Ce bucket conserve les données des dernières 24 heures, afin de rester utile pour d'autres scénarios (notamment pour faire les fonctions allégées des autres scénarios. [4.2.5](#))

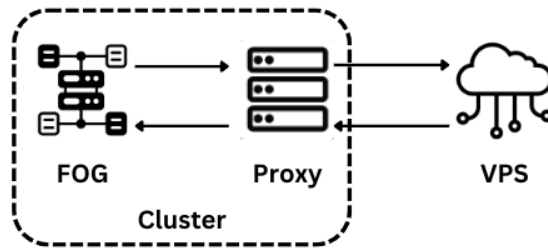


Figure 8: Schéma de l'architecture avec le proxy

- **Bucket d'erreur** : Ce bucket recueille les données qui n'ont pas pu être transférées au VPS en raison d'échecs de connexion.

Gestion des données dans le bucket de sauvegarde :

- **Cas 1** : La connexion est active et les données ont été transférées avec succès au VPS ; elles sont alors supprimées du bucket de sauvegarde.
- **Cas 2** : La connexion est active mais le transfert échoue ; les données passent dans le bucket d'erreur.
- **Cas 3** : La connexion est inactive ; les données sont transférées directement dans le bucket d'erreur.

4.2.3 Compression des données du bucket d'erreur

Supposons une interruption prolongée de la connexion entre le cluster et le VPS, pouvant durer plusieurs jours, voire semaines. Étant donné l'espace disque limité du cluster, le bucket d'erreur continuera de se remplir durant cette période. Afin de prévenir toute saturation, il sera essentiel de filtrer ces données de manière efficace.

Stratégie de filtrage appliquée : Les données qui n'apportent pas d'informations pertinentes seront remplacées par des indicateurs statistiques. (moyenne, minimum et maximum)

Ce filtrage, couplé à une compression régulière des données, permettra de maintenir une gestion optimale de la mémoire du cluster malgré l'accumulation des données non transférées.

4.2.4 Détection du retour de connexion et envoi des données

Ce scénario permet de reprendre le fonctionnement normal de l'architecture après une déconnexion. Une fois que le proxy détecte le retour de la connexion via ses pings, le transfert des données en attente vers le VPS est initié.

Algorithme d'envoi progressif :

- **Priorité aux données récentes** : Les données les plus récentes sont transférées en priorité pour maintenir la continuité des analyses.
- **Contrôle de débit** : Les données sont envoyées par petits lots pour éviter de saturer la connexion et les ressources.

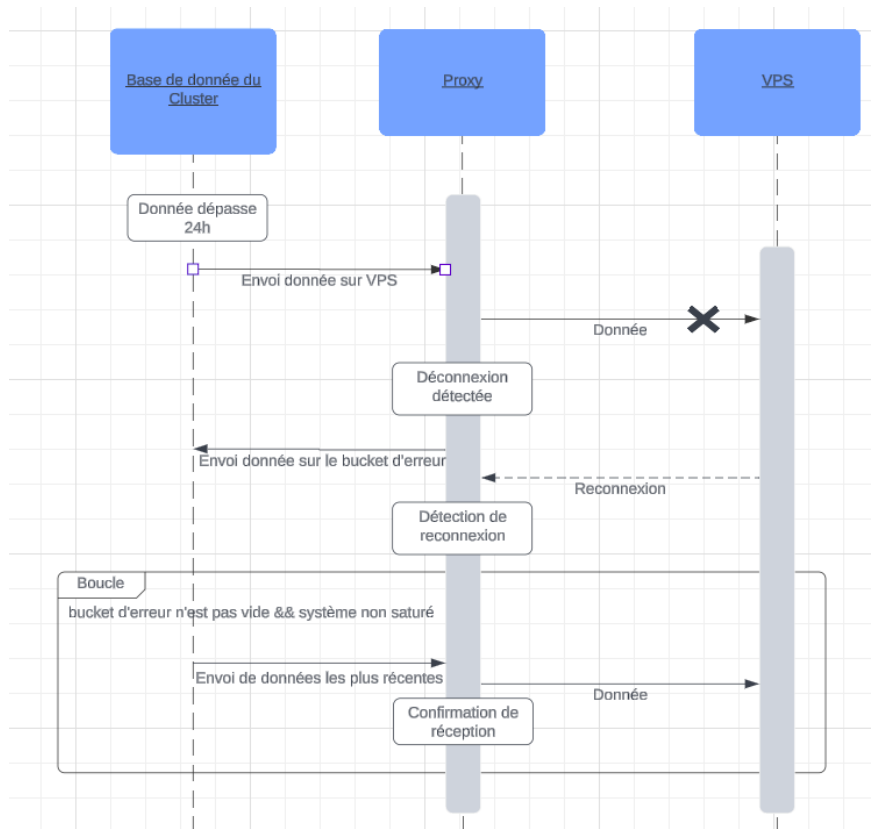


Figure 9: Diagramme de séquence du proxy pour l'envoi de donnée

4.2.5 Mode de Fonctionnement Allégé des Fonctions VPS sur le Cluster

Cette partie a pour but d'assurer le fonctionnement continu de l'architecture en cas de perte de connexion ou de ressources limitées sur le VPS. Le cluster exécute alors certaines fonctions du VPS en mode allégé, en adaptant leur consommation de ressources aux capacités du cluster.

Fonctionnalités allégées :

- **Calculs de changement de fréquence :** Un calcul prédictif restreint, basé sur les données des dernières 24 heures, estimera si une modification de la fréquence de capture des capteurs est nécessaire. On peut penser par exemple au scénario 4.1 Inondation, si les données de pluviométries du capteur dépasse un certain seuil alors on pourrait augmenter sa fréquence ou au contraire la réduire si elle passe en dessous d'un autre seuil.
- **Alertes locales :** Le cluster effectuera le travail de traitement sur les données du bucket d'erreur qui retournera une notification sur un flux RSS local. Au retour de la connexion entre le cluster et le VPS, celui-ci pourra récupérer pour mettre sur son flux RSS les notifications produites pendant la déconnexion.

Ce scénario permet de reprendre le fonctionnement normal de l'architecture après une déconnexion. Une fois que le proxy détecte le retour de la connexion via ses pings, le transfert des données en attente vers le VPS est initié.

4.3 Gestion dynamique de la fréquence des capteurs

Dans certains cas spécifiques, il peut être pertinent d'adapter la fréquence d'envoi des données directement au niveau des capteurs eux-mêmes. Cette modification permet d'économiser non seulement les ressources réseau et les capacités de traitement en aval, mais également l'énergie des capteurs, ce qui est crucial pour prolonger leur durée de vie en conditions d'utilisation intensive.

Voici quelques scénarios dans lesquels une adaptation de la fréquence est envisagée :

- **Conditions stables prévues** : Dans le cas de conditions météorologiques prévues comme étant stables (par exemple, absence de précipitations et températures normales), la fréquence d'envoi des relevés tels que la température ou l'humidité peut être réduite. Cela permet d'économiser de l'énergie sur les capteurs sans compromettre la qualité de la surveillance environnementale.
- **Risque accru d'événements significatifs ou variations soudaines des données** : En cas de prévision de phénomènes à risque (par exemple, un risque accru d'incendie en raison de températures élevées et d'une faible humidité) ou de détection de changements soudains dans les données collectées, il est nécessaire d'augmenter temporairement la fréquence des capteurs. Cette augmentation permet de capturer les variations en temps réel et de surveiller de manière proactive les zones à risque.

4.3.1 Gestion dynamique des capteurs en cas d'événements

La gestion de la fréquence des capteurs reposera sur un mécanisme d'adaptation aux événements critiques, intégrant une analyse locale et des calculs avancés via le proxy et le VPS.

Déroulement du processus :

- Les capteurs transmettront en continu leurs données au cluster, où une analyse locale surveillera les seuils critiques.
- En cas de détection d'un événement (par exemple, un risque de gelée), le flot normal de fonctionnement prévoit l'initiation d'une demande de calcul avancé via le proxy, qui sollicitera le VPS.
- Si le VPS est disponible, il exécutera des modèles prédictifs complexes (comme une estimation précise du risque de gel) et transmettra les résultats au cluster pour ajuster la fréquence des capteurs.
- Si le VPS n'est pas disponible, une analyse locale simplifiée sera effectuée directement sur le cluster en s'appuyant sur des seuils préconfigurés. Ces calculs locaux, bien que moins précis, permettront tout de même d'ajuster temporairement la fréquence des capteurs pour assurer une surveillance accrue.

Ce mécanisme garantira une surveillance adaptative tout en tenant compte des ressources disponibles.

4.3.2 Appel à des fonctions prédictives via un proxy

Le cluster intégrera un système de détection d'événements critiques, capable de déléguer certaines tâches analytiques complexes au VPS grâce à un proxy. Ce proxy jouera un rôle de proxy, assurant la communication entre le cluster et le VPS. Il permettra de transmettre les données nécessaires et de récupérer les résultats des calculs avancés.

Le proxy sera conçu pour gérer les connexions de manière optimisée, en tenant compte de la disponibilité des ressources du VPS. En cas d'indisponibilité, le cluster pourra basculer sur une analyse simplifiée en local, garantissant la continuité du service.

Exemple d'utilisation : Lorsqu'une chute rapide de température sera détectée par les capteurs, le proxy activera un modèle prédictif sur le VPS pour anticiper un épisode de gel. Les résultats de cette analyse seront utilisés pour ajuster dynamiquement la fréquence des capteurs et ainsi améliorer la réactivité du système face à cet événement critique.

Ce mécanisme assurera une répartition efficace des tâches entre le cluster et le VPS, tout en offrant une flexibilité et une robustesse accrues en cas de surcharge ou de défaillance des ressources distantes.

4.3.3 Exécution des fonctions de prévision allégées

En cas de surcharge du VPS, une version allégée des fonctions prédictives sera activée sur le cluster. Cette version reposera sur des seuils locaux simples pour maintenir une surveillance minimale.

Réaction à une gelée :

Lorsqu'une chute de température sera détectée localement, une augmentation immédiate de la fréquence des capteurs sera déclenchée, même si le VPS est inaccessible. Cela garantira une réactivité adaptée même avec des ressources limitées.

4.3.4 Schéma du processus

Le diagramme ci-dessous illustrera le processus complet d'ajustement dynamique de la fréquence des capteurs :

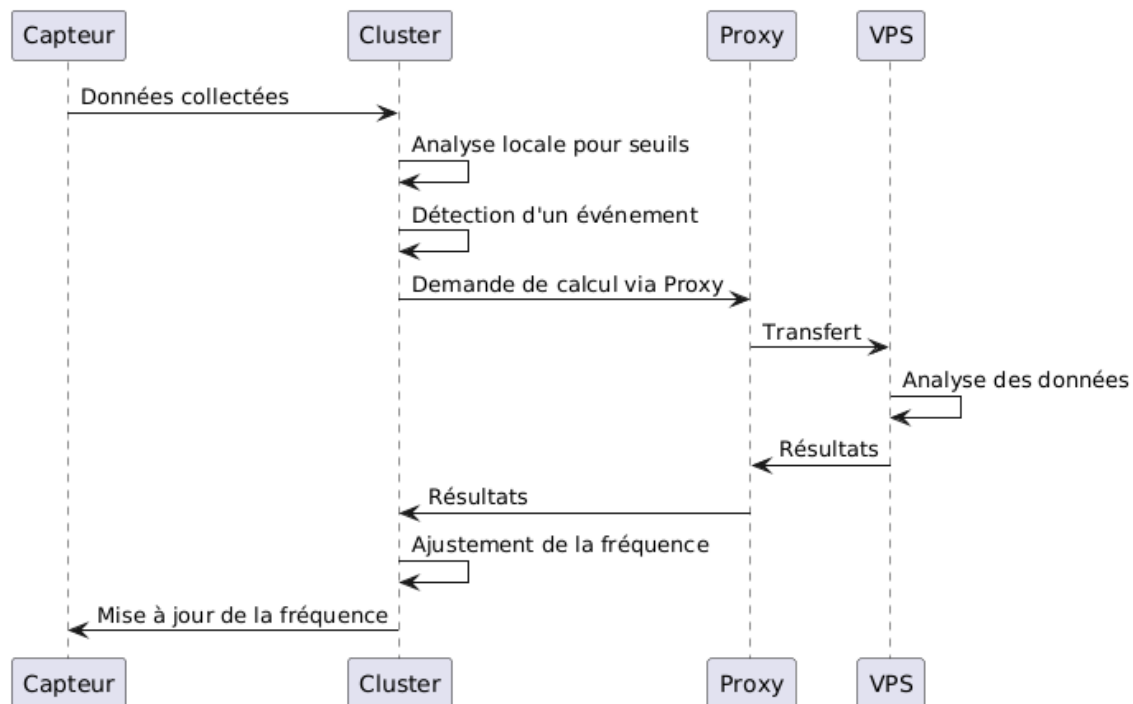


Figure 10: Diagramme de séquence pour la gestion dynamique de la fréquence des capteurs.

4.4 Mise en place d'un environnement virtuel simple

Le développement du projet repose principalement sur l'utilisation d'un cluster de Raspberry Pi. Pour exécuter les différents scénarios de test et valider les avancées, il est nécessaire de se connecter au cluster, que ce soit physiquement ou à distance via SSH. Cependant, cette approche pose plusieurs difficultés logistiques et techniques, qui peuvent limiter l'avancement du projet : par exemple, il est impossible de travailler à plusieurs en même temps, chaque personne essaye d'apporter ses modifications en même temps sur le même système.

Afin de contourner ces limitations, le groupe de travail précédent avait simulé le cluster en utilisant des machines virtuelles (VM) locales. Cette solution, bien qu'opérationnelle, a révélé plusieurs inconvénients notables :

- **Temps de configuration** : Les VM nécessitent des configurations initiales et des ajustements qui consomment du temps.
- **Récupération en cas de panne** : En cas de dysfonctionnement, il est souvent nécessaire de recommencer la configuration ou de restaurer une copie, avec un risque potentiel de perte de progression.
- **Partage des avancées** : Les résultats obtenus sur une VM ne sont pas facilement transférables ou partageables avec d'autres membres de l'équipe, ce qui peut nuire à la fluidité du travail en équipe.
- **Complexité de configuration** : La configuration d'une VM peut être laborieuse pour les nouveaux utilisateurs, ce qui représente un frein, exemple : installer docker n'est même pas simple sur ubuntu.

Adoption de Nix et Nix Flake pour une virtualisation optimisée

Suite aux recommandations de Mr. Parol-Guarino, nous avons opté pour une solution de virtualisation basée sur Nix, utilisant Nix Flake pour créer un environnement virtuel complet, isolé et reproductible, adapté aux besoins du projet. Grâce à cette configuration, il est possible de lancer une machine virtuelle "stateless" (sans état persistant) avec l'ensemble des outils nécessaires à l'aide d'une simple commande, sans modification du système hôte.

Détails techniques de la solution

- **Base de la configuration** : La configuration fournie par Mr. Parol-Guarino contient tous les éléments pour lancer une machine virtuelle via Nix, avec un terminal prêt à exécuter les logiciels requis, sans installation directe sur la machine hôte.
- **Gestion des dépendances** : Nix permet une gestion centralisée et précise des dépendances logicielles. Les logiciels nécessaires sont installés directement dans la machine virtuelle, de manière isolée, sans affecter le système hôte. Nix Flake est utilisé pour structurer et versionner les dépendances, facilitant ainsi le partage et les ajustements apportés à l'environnement tout au long du projet.
- **Personnalisation** : La configuration de base de Nix nécessite certains ajustements pour correspondre aux besoins spécifiques du projet. Par exemple, il est nécessaire d'intégrer des services tels qu'InfluxDB et Grafana. Pour cela, nous avons modifié la configuration afin que ces services soient lancés automatiquement au démarrage de la machine virtuelle, prêts à recevoir des données sans configuration supplémentaire.
- **Automatisation par alias** : Des alias de commandes permettent de simplifier le lancement de la machine virtuelle et l'accès aux différents services configurés. Un alias unique permet de démarrer l'environnement complet, réduisant ainsi les étapes de configuration manuelles pour chaque session de test.

Simplification de la stack réseau

Notre objectif ici était de simplifier la connexion entre les différents systèmes du projet pour éviter que le projet soit bloqué à cause de problèmes réseaux. Nous avons donc connecté le VPS, le cluster ainsi que nos machines sur Tailscale, désormais nous pouvons accéder au VPS ou au cluster simplement en nous connectant sur le VPN de Tailscale.

Avantages pour le développement collaboratif

Ce système de virtualisation via Nix assure un environnement de développement homogène pour tous les membres de l'équipe, éliminant les différences de configuration qui pourraient causer des erreurs. De plus, il favorise une approche durable de déploiement continu, en permettant des tests rapides et fiables dans un environnement proche de celui de production. Cela minimise les risques et allège la charge de configuration

```
Starting User Login Management...
Starting Permit User Sessions...
[ OK ] Finished Permit User Sessions.
[ OK ] Started Getty on tty1.
[ OK ] Started Serial Getty on ttyS0.
[ OK ] Reached target Login Prompts.
[ OK ] Started D-Bus System Message Bus.
[ OK ] Started User Login Management.
[ OK ] Started Mosquitto MQTT Broker Daemon.
[ OK ] Started SSH Daemon.
[ OK ] Started InfluxDB is an open-source, distributed, time series database.
Starting Configuration initiale d'InfluxDB...
[ OK ] Finished Configuration initiale d'InfluxDB.

<<< Welcome to NixOS 24.11.20241023.c091517 (x86_64) - ttyS0 >>>

localhost login: myce (automatic login)

[myce@localhost:~]$
```

Figure 11: Exemple de machine virtuelle sans état démarrée avec la commande *just vm*

initiale, facilitant la prise en main pour les nouveaux développeurs et garantissant une continuité dans le projet. Pour l'instant nous conservons Ubuntu en production et Nix en développement mais il serait judicieux dans le futur d'unifier les deux pour simplifier et automatiser les déploiements et la gestion du cluster.

4.5 Le proxy dans Mycelium 4.0

Le proxy est un élément central dans l'architecture du projet Mycelium 4.0, jouant un rôle crucial dans les trois scénarios prévus pour cette année. Afin de clarifier son fonctionnement, souvent évoqué dans diverses sections du rapport, cette sous-section propose une synthèse globale des fonctions du proxy, avec des références appropriées pour en faciliter la compréhension.

Détection de la Perte de Connexion :

Comme indiqué dans la section [4.2.1](#), le proxy assure un suivi continu de la connexion entre le VPS et le cluster. Il effectue des pings réguliers et signale toute interruption. Cette fonctionnalité est essentielle pour déclencher des actions compensatoires, telles que le basculement du mode de fonctionnement du cluster.

Gestion en Cas de Déconnexion Prolongée :

Lorsque la connexion entre le VPS et le cluster est interrompue, le proxy active des mécanismes pour éviter toute perte de données, comme décrit en détail dans [4.2.2](#) et [4.2.3](#), ces mécanismes incluent :

- Compression et Filtrage des Données : Les données stockées localement dans le bucket d'erreur sont filtrées et compressées pour économiser l'espace disque du cluster.
- Traitement Local et Notifications : Pendant la déconnexion, le proxy peut générer des résumés sous forme de notifications RSS, lesquelles seront transmises au VPS dès la reconnexion. Voir [4.2.5](#).

Adaptation Dynamique du Comportement Système :

Le proxy joue également un rôle clé dans l'adaptation du comportement du système en fonction de la disponibilité des ressources et de l'état de la connexion. On peut distinguer plusieurs cas possibles aux

proxy pour effectuer ses calculs de prédictions :

- Si le VPS est connecté et dispose de suffisamment de ressources, le proxy délègue des calculs prédictifs complexes au VPS. [4.3.2](#)
- Si le VPS est surchargé ou déconnecté, une version allégée des calculs est exécutée directement sur le cluster. [4.2.5](#)

Ces ajustements garantissent un fonctionnement optimal, que le VPS soit actif ou non.

Transition et Réintégration des Données après Reconnexion :

Dès que la connexion est rétablie, le proxy initie un transfert progressif des données accumulées, comme précisé en [4.2.4](#). Les priorités de ce processus incluent :

- L'envoi des données récentes en premier.
- Un contrôle de débit pour éviter une surcharge réseau.

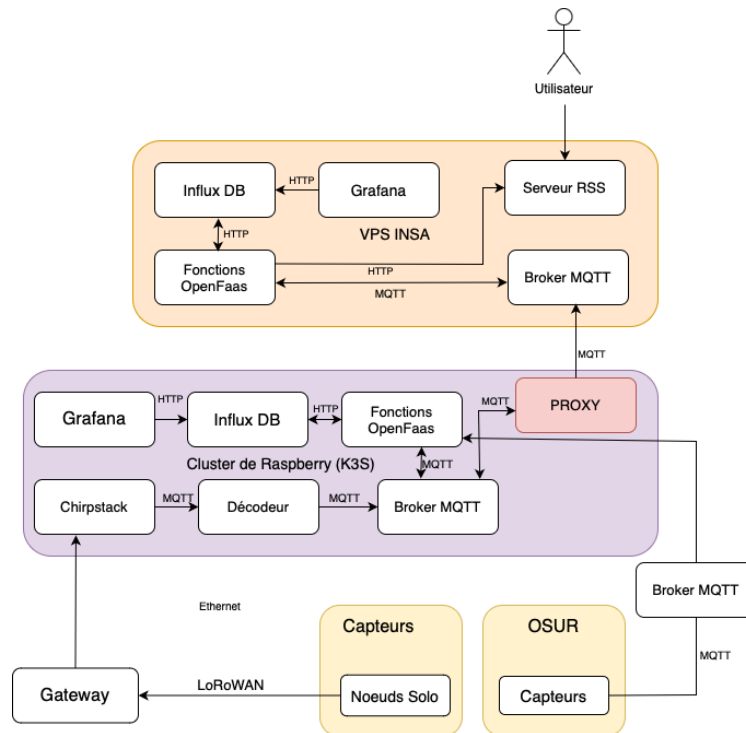


Figure 12: Schéma de l'architecture 4.0 avec le proxy.

Schéma de l'architecture Mycelium 3.0 : [3.2](#)

En regroupant ces informations, cette synthèse vise à centraliser la compréhension des contributions du proxy au projet Mycelium 4.0.

5 Conclusion

Le projet Mycélium vise donc à la gestion de la **renaturalisation** du site de la croix verte (Beaulieu, Rennes), via un déploiement de capteurs, et d'une infrastructure capable de **traiter efficacement et durablement** leurs acquisitions. Les équipes d'étudiants (du département informatique de l'INSA Rennes) qui travaillent sur le projet ont ainsi pour mission de mettre en place cette infrastructure.

Malgré un aspect matériel complet, le projet a connu quelques restructurations majeures, et ne présente donc pas encore les **implémentations logicielles** permettant un **fonctionnement et une intégration continue**. L'objectif de notre groupe, pour le projet Mycélium 4.0, sera alors de réaliser ces implémentations en se basant sur l'architecture et les technologies de la version 3.0 (Fog computing, FaaS, LoWaRAN, ...).

Ce document présente donc d'abord l'état du projet Mycélium à sa version 3.0, dans son architecture **matérielle** (capteurs, nœuds, clusters Raspberry, ...) et **logicielle** (technologies, scénarios, stratégies de stockage et de communication...). Il introduit ensuite nos **projections** pour la version 4.0 (adaptation de l'architecture logicielle, nouveaux scénarios, mise en place globale de l'environnement virtuel de développement...).

Ces projections ont donc pour but de permettre **un fonctionnement continu et durable** de l'infrastructure, même en cas de perte de connexion (au VPS) ou de surcharge (émissions des capteurs) du cluster. Cette amélioration passera par la création de scénarios critiques (perte de connexion, changement de fréquence d'émission des capteurs), le stockage dans une base de données locale (durant la déconnexion), et l'utilisation d'un proxy (pour détecter et prendre en charge la déconnexion).

Mais ces projections ont aussi pour but de permettre **un déploiement continu** sur l'architecture matérielle en place, facilitant le développement de scénarios, et intégrant la phase de test avant le build. Pour se faire, il faudra créer l'environnement virtuel de développement, simulant le cluster RaspBerry et ses entrées d'acquisition des capteurs. Il faudra aussi adapter les communications et le système de notification à la VM (Nats → MQTT, Discord → RSS). Enfin nous passerons sur K3S pour faciliter la communication entre les nœuds, et le traitement des données en temps réel. Les fonctions développées dans cet environnement devront alors fonctionner directement sur le cluster réel.

Via ces ajouts et modifications, nous devrions être capable de proposer une infrastructure **stable, autonome et durable**, avec un environnement de développement plus **efficace** et **agréable**. Ces améliorations permettraient **d'accélérer** grandement la croissance du projet, et peut-être, d'ouvrir des perspectives à plus grande échelle via TERRA FORMA et les projets qui lui succéderont.